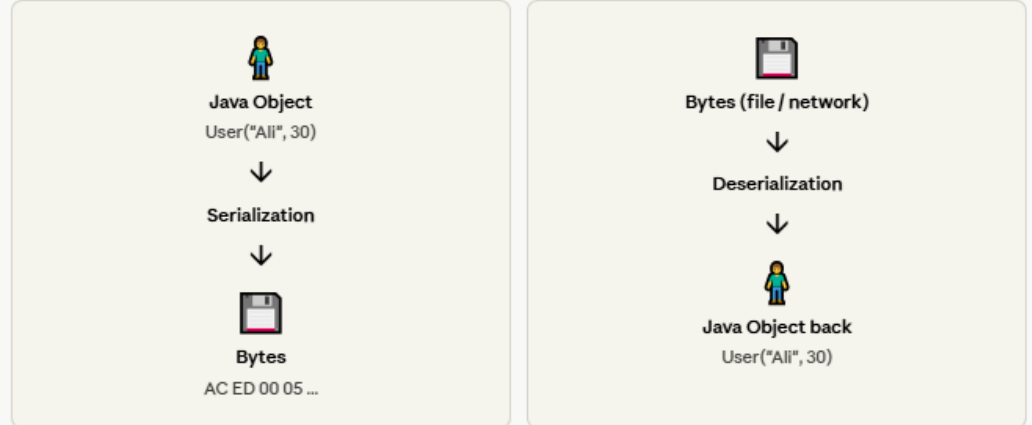
Serialization et Deserialization



Java built-in serialization — Serializable interface

```java
// Step 1 — implement Serializable (marker interface)
import java.io.Serializable;

class User implements Serializable {
    private static final long serialVersionUID = 1L; // version stamp
    private String name;
    private int age;

    User(String name, int age) {
        this.name = name;
        this.age  = age;
    }
}

// Step 2 — SERIALIZE: object → file
User user = new User("Ali", 30);

try (ObjectOutputStream oos =
        new ObjectOutputStream(
            new FileOutputStream("user.ser"))) {
    oos.writeObject(user);           // packed into bytes → file
}

// Step 3 — DESERIALIZE: file → object
try (ObjectInputStream ois =
        new ObjectInputStream(
            new FileInputStream("user.ser"))) {
    User loaded = (User) ois.readObject(); // bytes → object back
    System.out.println(loaded.name);       // "Ali"
}
```

Excluding fields — the transient keyword

Some fields should NOT be serialized: passwords, connections, Optional fields, computed values.

```java
class User implements Serializable {
    private static final long serialVersionUID = 1L;
```

```
    String name;
    transient String    password;    // excluded — security
    transient Optional<String> email; // excluded — not Serializable!
    transient Connection db;         // excluded — makes no sense
}

User user = new User();
user.name     = "Ali";
user.password = "secret123";

// After serialize → deserialize:
loaded.name     → "Ali"      // saved
loaded.password → null       // transient: wiped
loaded.email    → null       // transient: wiped
```

ℹ This is exactly why `Optional` cannot be a field — Optional does not implement Serializable. Mark it `transient` or use a nullable field instead.

Serialization = Java emballe TOUS les champs dans des bytes.

```
class User implements Serializable {
    String name;           // String  → Serializable  Java sait emballer
    int    age;            // int     → Serializable  Java sait emballer
    Optional<String> email; // Optional → ???  Java ne sait pas emballer
}

// Ce code plante à l'exécution :
User user = new User("Ali", 30, Optional.of("ali@gmail.com"));
new ObjectOutputStream(...).writeObject(user); // NotSerializableException: Optional
```

La solution — séparer le stockage de l'accès :

```
class User implements Serializable {

    String name;
    int    age;
    String email;        //champ nullable — Java sait l'emballer

    // Optional UNIQUEMENT dans le getter — jamais stocké
    Optional<String> getEmail() {
        return Optional.ofNullable(email);  // créé à la volée
    }
}
```

Java built-in serialization is rare — JSON is what you use daily

REST APIs, Spring Boot, microservices → everything is JSON serialization via Jackson.

```
// Jackson dependency (Maven / Spring Boot already includes it)
// <dependency> com.fasterxml.jackson.core:jackson-databind </dependency>

import com.fasterxml.jackson.databind.ObjectMapper;

class User {
    public String name;
    public int    age;
```

```
    User() {}                          // Jackson needs no-arg constructor
    User(String n, int a) { name=n; age=a; }
}

ObjectMapper mapper = new ObjectMapper();

// SERIALIZE: object → JSON string
User user = new User("Ali", 30);
String json = mapper.writeValueAsString(user);
// → {"name":"Ali","age":30}

// DESERIALIZE: JSON string → object
User loaded = mapper.readValue(json, User.class);
// → User("Ali", 30)

// File ↔ object
mapper.writeValue(new File("user.json"), user);
User fromFile = mapper.readValue(new File("user.json"), User.class);
```

Jackson annotations — what you'll use in Spring Boot daily

```
import com.fasterxml.jackson.annotation.*;

class User {
    // rename field in JSON
    @JsonProperty("full_name")
    public String name;

    // exclude from JSON output
    @JsonIgnore
    public String password;

    // include only when not null
    @JsonInclude(JsonInclude.Include.NON_NULL)
    public String email;

    // format dates
    @JsonFormat(pattern = "dd/MM/yyyy")
    public LocalDate birthDate;
}

// Result:
// { "full_name": "Ali", "birthDate": "01/01/1994" }
// password → gone (JsonIgnore)
// email    → gone (null + NON_NULL)
```

In Spring Boot — serialization is automatic

```
// You write this:
@RestController
class UserController {

    @GetMapping("/user/{id}")
    User getUser(@PathVariable int id) {
        return new User("Ali", 30);   // return object
    }                                  // Spring serializes → JSON

    @PostMapping("/user")
    User createUser(@RequestBody User user) {  // JSON → object
        return userService.save(user);         // auto deserialized
    }
}
```

```
// HTTP GET /user/1  →  {"name":"Ali","age":30}
// HTTP POST /user   ←  {"name":"Sara","age":25}
```

Spring Boot uses Jackson automatically — you never call `ObjectMapper` manually in controllers.